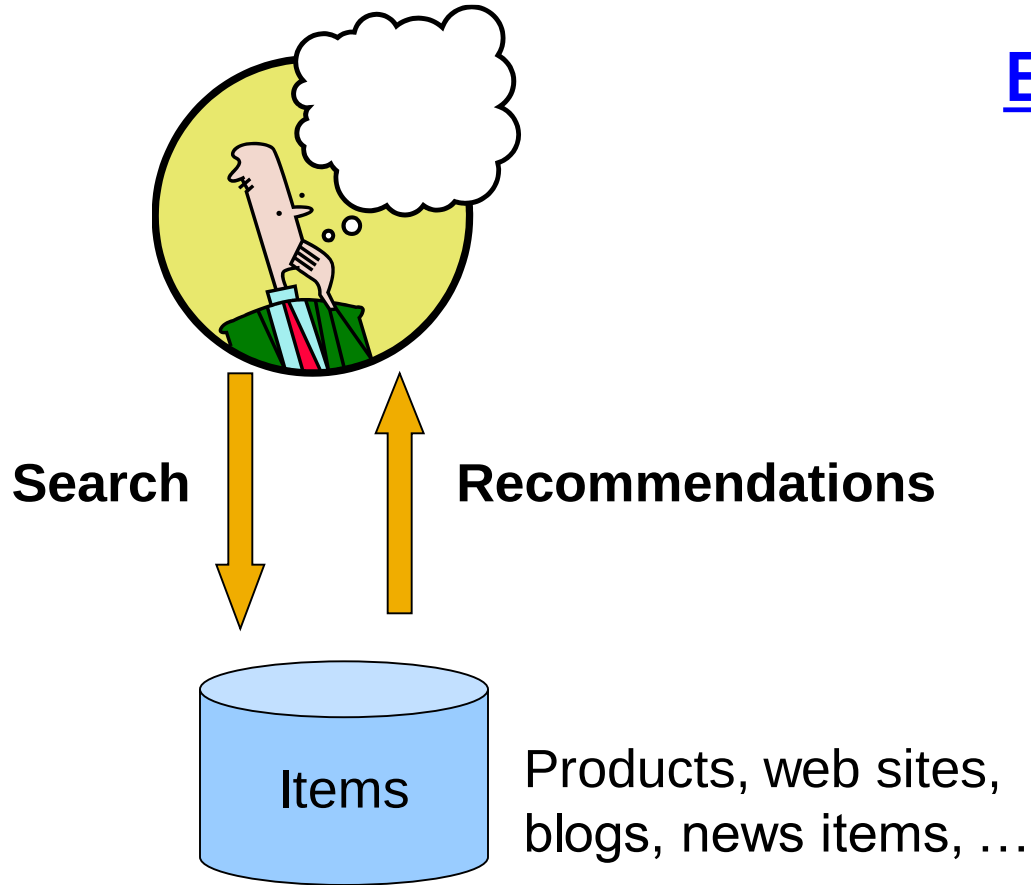


Recommender Systems

BY

N.Omkar

Recommendations



Examples:

amazon.com.



m o v i e l e n s
helping you find the *right* movies

last.fm™
the social music revolution

Google
News

You Tube

XBOX
LIVE

From Scarcity to Abundance

- **Shelf space is a scarce commodity for traditional retailers**
 - Also: TV networks, movie theaters,...
- **Web enables near-zero-cost dissemination of information about products**
 - From scarcity to abundance
- **More choice necessitates better filters**
 - Recommendation engines
 - How **Into Thin Air** made **Touching the Void** a bestseller

Popularity

**Retail &
online**

Only in Online

Products

The “head”

The Long Tail - niche products

Formal Model

- X = set of **Customers**
- S = set of **Items**
- **Utility function** $u: X \times S \rightarrow R$
 - R = set of ratings
 - R is a totally ordered set
 - e.g., **0-5** stars, real number in **[0,1]**

Utility Matrix

	Bahubali	Dhanganal	Robo	MI
Anand	1		0.2	
Vinayak		0.5		0.3
Manzil	0.2		1	
Omkar				0.4

Key Problems

- **(1) Gathering “known” ratings for matrix**
 - How to collect the data in the utility matrix
- **(2) Extrapolate unknown ratings from the known ones**
 - Mainly interested in high unknown ratings
 - We are not interested in knowing what you don't like but what you like
- **(3) Evaluating extrapolation methods**
 - How to measure success/performance of recommendation methods

(1) Gathering Ratings

■ Explicit

- Ask people to rate items
- Doesn't work well in practice – people can't be bothered

■ Implicit

- Learn ratings from user actions
 - E.g., purchase implies high rating
- What about low ratings?

(2) Extrapolating Utilities

- **Key problem:** Utility matrix U is **sparse**
 - Most people have not rated most items
 - **Cold start:**
 - New items have no ratings
 - New users have no history
- **Three approaches to recommender systems:**
 - 1) Content-based
 - 2) Collaborative
 - 3) Latent factor based

Content-based Recommender Systems

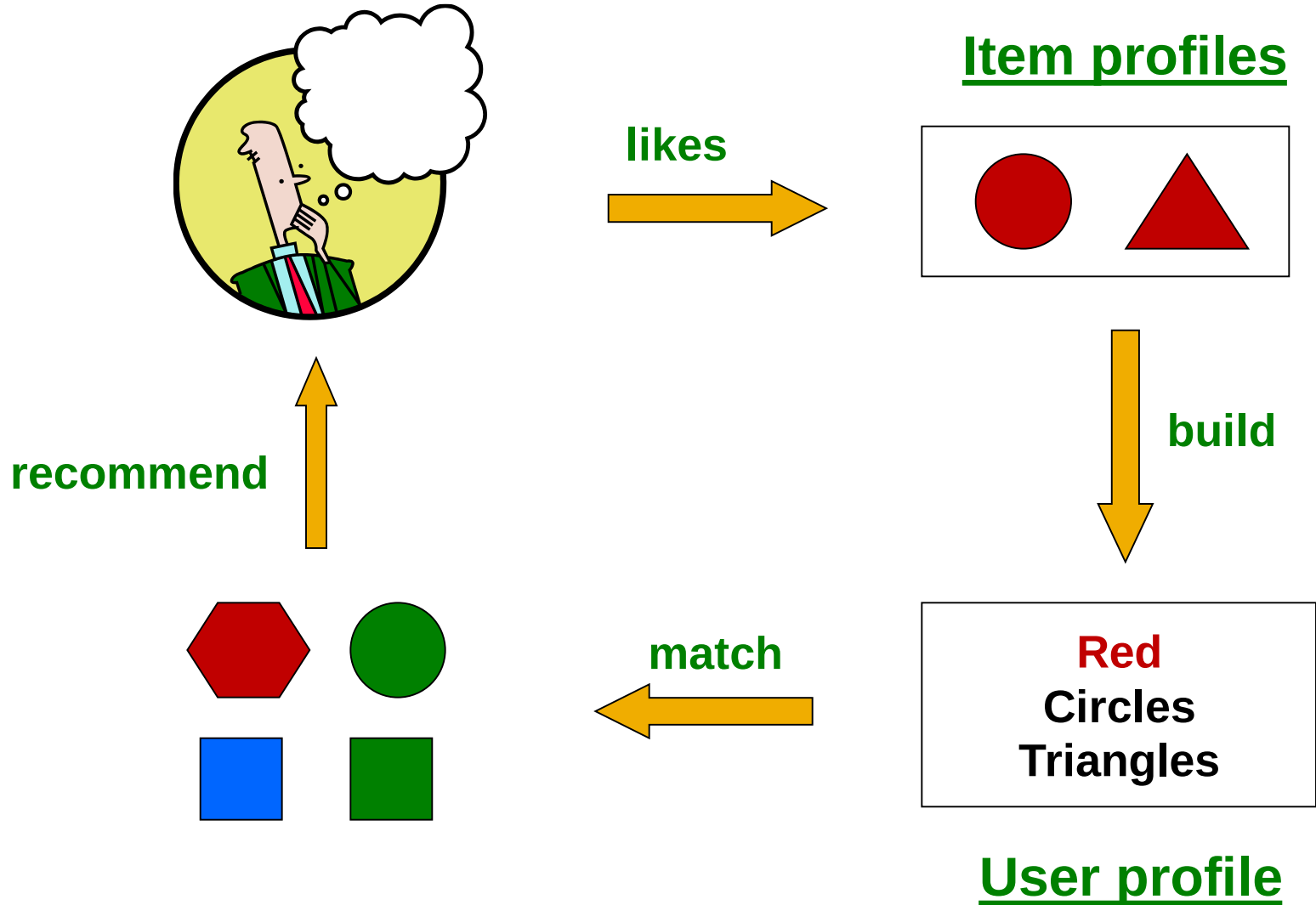
Content-based Recommendations

- **Main idea:** Recommend items to customer x similar to previous items rated highly by x

Example:

- **Movie recommendations**
 - Recommend movies with same actor(s), director, genre, ...
- **Websites, blogs, news**
 - Recommend other sites with “similar” content

Plan of Action



Item Profiles

- For each item, create an **item profile**
- **Profile is a set (vector) of features**
 - **Movies:** author, title, actor, director,...
 - **Text:** Set of “important” words in document

User Profiles and Prediction

- **User profile possibilities:**

- Weighted average of rated item profiles
- **Variation:** weight by difference from average rating for item
- ...

- **Prediction heuristic:**

- Given user profile $\mathbf{x}$ and item profile $\mathbf{i}$, estimate

$$u(\mathbf{x}, \mathbf{i}) = \cos(\mathbf{x}, \mathbf{i}) = \frac{\mathbf{x} \cdot \mathbf{i}}{||\mathbf{x}|| \cdot ||\mathbf{i}||}$$

Pros: Content-based Approach

- **+: No need for data on other users**
 - No cold-start or sparsity problems
- **+: Able to recommend to users with unique tastes**
- **+: Able to recommend new & unpopular items**
 - No first-rater problem
- **+: Able to provide explanations**
 - Can provide explanations of recommended items by listing content-features that caused an item to be recommended

Cons: Content-based Approach

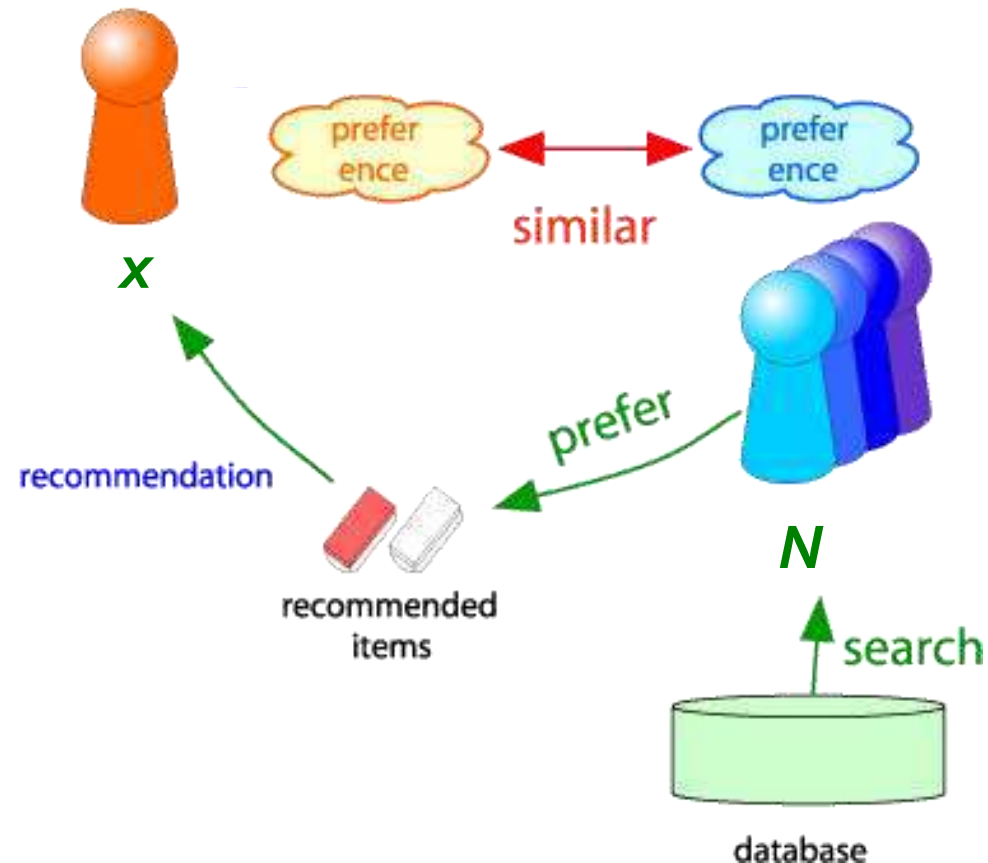
- —: Finding the appropriate features is hard
 - E.g., images, movies, music
- —: Recommendations for new users
 - How to build a user profile?
- —: Overspecialization
 - Never recommends items outside user's content profile
 - People might have multiple interests
 - Unable to exploit quality judgments of other users

Collaborative Filtering

Harnessing quality judgments of other users

Collaborative Filtering

- Consider user x
- Find set N of other users whose ratings are “**similar**” to x ’s ratings
- Estimate x ’s ratings based on ratings of users in N



Finding “Similar” Users

$$r_x = \begin{bmatrix} *, & _, & _, & *, & *** \end{bmatrix}$$

$$r_y = \begin{bmatrix} *, & _, & **, & **, & _ \end{bmatrix}$$

- Let r_x be the vector of user x 's ratings

- Jaccard similarity measure**

- Problem:** Ignores the value of the rating

r_x, r_y as sets:

$$r_x = \{1, 4, 5\}$$

$$r_y = \{1, 3, 4\}$$

- Cosine similarity measure**

- $\text{sim}(x, y) = \cos(r_x, r_y) = \frac{r_x \cdot r_y}{\|r_x\| \cdot \|r_y\|}$

r_x, r_y as points:

$$r_x = \{1, 0, 0, 1, 3\}$$

$$r_y = \{1, 0, 2, 2, 0\}$$

- Problem:** Treats missing ratings as “negative”

- Pearson correlation coefficient**

- S_{xy} = items rated by both users x and y

$$\text{sim}(x, y) = \frac{\sum_{s \in S_{xy}} (r_{xs} - \bar{r}_x)(r_{ys} - \bar{r}_y)}{\sqrt{\sum_{s \in S_{xy}} (r_{xs} - \bar{r}_x)^2} \sqrt{\sum_{s \in S_{xy}} (r_{ys} - \bar{r}_y)^2}}$$

$\bar{r}_x, \bar{r}_y \dots$ avg.
rating of x, y

Similarity Metric

$$\text{Cosine sim: } \text{sim}(x, y) = \frac{\sum_i r_{xi} \cdot r_{yi}}{\sqrt{\sum_i r_{xi}^2} \cdot \sqrt{\sum_i r_{yi}^2}}$$

	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	4			5	1		
B	5	5	4				
C				2	4	5	
D		3					3

- **Intuitively we want:** $\text{sim}(A, B) > \text{sim}(A, C)$
- **Jaccard similarity:** $1/5 < 2/4$
- **Cosine similarity:** $0.386 > 0.322$
 - Considers missing ratings as “negative”

- **Solution: subtract the (row) mean**

	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	2/3			5/3	-7/3		
B	1/3	1/3	-2/3				
C				-5/3	1/3	4/3	
D		0					0

sim A,B vs. A,C:
 $0.092 > -0.559$

Notice cosine sim. is correlation when data is centered at 0

Rating Predictions

From similarity metric to recommendations:

- Let $\mathbf{r}_x$ be the vector of user x 's ratings
- Let N be the set of k users most similar to x who have rated item i
- **Prediction for item s of user x :**

- $r_{xi} = \frac{1}{k} \sum_{y \in N} r_{yi}$

- $r_{xi} = \frac{\sum_{y \in N} s_{xy} \cdot r_{yi}}{\sum_{y \in N} s_{xy}}$

Shorthand:

$$s_{xy} = \text{sim}(x, y)$$

Item-Item Collaborative Filtering

- So far: **User-user collaborative filtering**
- **Another view: Item-item**
 - For item i , find other similar items
 - Estimate rating for item i based on ratings for similar items
 - Can use same similarity metrics and prediction functions as in user-user model

$$r_{xi} = \frac{\sum_{j \in N(i; x)} s_{ij} \cdot r_{xj}}{\sum_{j \in N(i; x)} s_{ij}}$$

s_{ij} ... similarity of items i and j
 r_{xj} ... rating of user u on item j
 $N(i; x)$... set items rated by x similar to i

Item-Item CF ($|N|=2$)

		users											
		1	2	3	4	5	6	7	8	9	10	11	12
movies	1	1		3			5			5		4	
	2			5	4			4			2	1	3
	3	2	4		1	2		3		4	3	5	
	4		2	4		5			4			2	
	5			4	3	4	2					2	5
	6	1		3		3			2			4	



- unknown rating



- rating between 1 to 5

Item-Item CF ($|N|=2$)

		users											
		1	2	3	4	5	6	7	8	9	10	11	12
movies	1	1		3		?	5			5		4	
	2			5	4			4			2	1	3
	3	2	4		1	2		3		4	3	5	
	4		2	4		5			4			2	
	5			4	3	4	2					2	5
	6	1		3		3			2			4	



- estimate rating of movie 1 by user 5

Item-Item CF ($|N|=2$)

		users												
		1	2	3	4	5	6	7	8	9	10	11	12	sim(1,m)
movies	1	1		3		?	5			5		4		1.00
	2			5	4			4			2	1	3	-0.18
	<u>3</u>	2	4		1	2		3		4	3	5		<u>0.41</u>
	4		2	4		5			4			2		-0.10
	5			4	3	4	2					2	5	-0.31
	<u>6</u>	1		3		3			2			4		<u>0.59</u>

Neighbor selection:

Identify movies similar to movie **1**, rated by user **5**

Here we use Pearson correlation as similarity:

1) Subtract mean rating m_i from each movie i

$$m_1 = (1+3+5+5+4)/5 = 3.6$$

row 1: [-2.6, 0, -0.6, 0, 0, 1.4, 0, 0, 1.4, 0, 0.4, 0]

2) Compute cosine similarities between rows

Item-Item CF ($|N|=2$)

		users												sim(1,m)
		1	2	3	4	5	6	7	8	9	10	11	12	
movies	1	1		3		?	5			5		4		1.00
	2			5	4			4			2	1	3	-0.18
	<u>3</u>	2	4		1	2		3		4	3	5		<u>0.41</u>
	4		2	4		5			4			2		-0.10
	5			4	3	4	2					2	5	-0.31
	<u>6</u>	1		3		3			2			4		<u>0.59</u>

Compute similarity weights:

$s_{1,3}=0.41$, $s_{1,6}=0.59$

Item-Item CF ($|N|=2$)

		users											
		1	2	3	4	5	6	7	8	9	10	11	12
movies	1	1		3		2.6	5			5		4	
	2			5	4			4			2	1	3
	<u>3</u>	2	4		1	2		3		4	3	5	
	4		2	4		5			4			2	
	5			4	3	4	2					2	5
	<u>6</u>	1		3		3			2			4	

Predict by taking weighted average:

$$r_{1.5} = (0.41 \cdot 2 + 0.59 \cdot 3) / (0.41 + 0.59) = 2.6$$

$$r_{ix} = \frac{\sum_{j \in N(i;x)} s_{ij} \cdot r_{jx}}{\sum s_{ij}}$$

CF: Common Practice

Before:

$$r_{xi} = \frac{\sum_{j \in N(i; x)} s_{ij} r_{xj}}{\sum_{j \in N(i; x)} s_{ij}}$$

- Define **similarity** s_{ij} of items i and j
- Select k nearest neighbors $N(i; x)$
 - Items most similar to i , that were rated by x
- Estimate rating r_{xi} as the weighted average:

$$r_{xi} = b_{xi} + \frac{\sum_{j \in N(i; x)} s_{ij} \cdot (r_{xj} - b_{xj})}{\sum_{j \in N(i; x)} s_{ij}}$$

baseline estimate for r_{xi}

$$b_{xi} = \mu + b_x + b_i$$

- μ = overall mean movie rating
- b_x = rating deviation of user x
= (avg. rating of user x) - μ
- b_i = rating deviation of movie i

Item-Item vs. User-User

	Avatar	LOTR	Matrix	Pirates
Alice	1		0.8	
Bob		0.5		0.3
Carol	0.9		1	0.8
David			1	0.4

- In practice, it has been observed that item-item often works better than user-user
- **Why?** Items are simpler, users have multiple tastes

Pros/Cons of Collaborative Filtering

- **+ Works for any kind of item**
 - No feature selection needed
- **- Cold Start:**
 - Need enough users in the system to find a match
- **- Sparsity:**
 - The user/ratings matrix is sparse
 - Hard to find users that have rated the same items
- **- First rater:**
 - Cannot recommend an item that has not been previously rated
 - New items, Esoteric items
- **- Popularity bias:**
 - Cannot recommend items to someone with unique taste
 - Tends to recommend popular items

Python code using SVD,KNN,NFM

Here I am Using Movie lense data set, my main objective is

- 1) Finding the Unknown Ratings
- 2) Increase The model Accuracy
- 3) I use Three approaches (i) Singular Value Decomposition (ii) K-Nearest Neighbours
- 4) (iii) Non negative Matrix Factorization

Performance:

RMSE for SVD=0.30

KNN=0.25

NFM=0.38

KNN Gives best Performance Lowest Root Mean Square Error

```
import numpy as np
import pandas as pd
```

```
_cols= ['user_id', 'age', 'gender', 'occupation', 'zip_code']
```

```
users=pd.read_csv('C:\\Users\\omkar nallagoni\\Desktop\\users.csv')
```

```
_cols= ['user_id', 'movie_id', 'rating', 'unix_timestamp']
```

```
ratings=pd.read_csv("C:\\Users\\omkar nallagoni\\Desktop\\ratings.csv")
```

```
i_cols = ['movie id', 'movie title', 'release date',  
          'Animation', 'Children\\s', 'Comedy', 'Crime', 'Drama',  
          'Fantasy', 'Film-Noir', 'Horror', 'Musical', 'Romance',  
          'Sci-Fi', 'TV-14', 'TV-MA', 'TV-PG', 'TV-Y7-FV',  
          'Unrated-NC-17', 'Unrated-R']  
movies = pd.read_csv('C:\\Users\\omkar nallagoni\\Desktop\\movies.csv')
```

users:

`users.shape`
(943, 5)

`users.head()` *#total 943 users, 5 features*

	user_id	age	gender	occupation	zip_code
0	1	24	M	technician	85711
1	2	53	F	other	94043
2	3	23	M	writer	32067
3	4	24	M	technician	43537
4	5	23	F	other	15212

Ratings

```
ratings.shape
```

```
(100000, 4)
```

```
ratings.head() #100k users, ratings with timestamp
```

	user_id	movie_id	rating	unix_timestamp
0	196	242	3	881250949
1	186	302	3	891717742
2	22	377	1	878887116
3	244	51	2	880606923
4	166	346	1	886397596

References:

- 1) Stanford University
- 2) Mining of moving Data sets
- 3) Andrew NG Coursera
- 4) Google Images

Thank You

